ASSIGNMENT 2 – CO2

Q6: PRIORITY, ORDER AND FAIRNESS IN A QUEUE

Hospital Emergency Triage

This report intentionally excludes student-identifying details.

1. Problem and design requirements

Patients are assigned severity 1 (critical) through 5 (minor). The next patient must be the most severe waiting patient; if severity is equal, the earlier arrival is served first. A plain FIFO Queue cannot satisfy this rule when a later arrival is more severe than patients already waiting.

2. PriorityQueue design

The solution uses PriorityQueue<Patient> with a Comparator that compares severity in ascending numeric order, then arrival sequence in ascending order. Severity 1 therefore has the highest priority. A monotonically increasing counter is stored in Patient.arrival. For equal severity, the smaller arrival value compares first, preserving FIFO order within that severity. Adding and removing are O(log n), peek is O(1), and storage is O(n).

3. Correctness check

The eight-patient hand-check dataset is Ravi(S3), Meena(S1), John(S3), Asha(S5), Kumar(S1), Lata(S2), Nila(S2), Arun(S4). Expected treatment order: Meena(S1), Kumar(S1), Lata(S2), Nila(S2), Ravi(S3), John(S3), Arun(S4), Asha(S5). This checks severity ordering and arrival order within equal severity. The exact six-patient assignment example is also implemented; expected output: Meena(S1) Kumar(S1) Lata(S2) Ravi(S3) John(S3) Asha(S5).

4. Waiting-list display

Printing PriorityQueue directly does not guarantee treatment order. It is a binary heap: the head is the least element according to the comparator, but the iterator and toString representation do not sort all remaining elements. The program copies the queue and repeatedly calls poll() on the copy to display the waiting list in treatment order without changing the live queue.

5. Growth prediction and experiment

Prediction: PriorityQueue admission and removal are O(log n). A sorted LinkedList needs O(n) admission to find the insertion point and O(1) removal from its head. Five FIFO queues provide O(1) admission and O(1) serving because the number of severity buckets is fixed at five; at most five queue heads are checked.

Strategy	Patients	Avg admit (ms)	Avg serve (ms)
PriorityQueue	500	0.1758	0.5136
PriorityQueue	2,000	0.2311	0.4284
PriorityQueue	5,000	0.2070	1.0218
SortedLinkedList	500	1.3026	0.0310
SortedLinkedList	2,000	6.2063	0.0912
SortedLinkedList	5,000	24.2890	0.0672
FiveFIFOQueues	500	0.0843	0.1087
FiveFIFOQueues	2,000	0.2615	0.0866
FiveFIFOQueues	5,000	0.1539	0.0365

The benchmark measures 500, 2,000 and 5,000 patients, with five repetitions per strategy and size. Admission and serving times are recorded separately with System.nanoTime(). Raw data is in task6_results.csv. The same seeded workload is used for each strategy. Timings depend on JVM, hardware, garbage collection and system load; regenerate results on the submission computer and interpret repeated measurements alongside Big-O analysis.

6. Alternative concept and complexity analysis

Sorted LinkedList: keep patients in comparator order. Admission scans for the correct position and inserts there, O(n); removing the first patient is O(1). Space is O(n). It shifts work to admission and may be costly during arrival bursts.

Five FIFO queues: maintain one FIFO queue per fixed severity 1–5. Admission enters its matching queue in O(1). Serving checks severity queues from 1 to 5 and removes from the first non-empty queue. With five fixed levels, the scan is O(1); total space is O(n) plus five queue objects. It naturally preserves arrival order within a severity.

7. Starvation and fairness

Under strict priority, a severity-5 patient can wait indefinitely if higher-severity patients continue arriving. This is starvation. A possible aging policy improves a patient's effective severity by one level after each defined waiting interval, capped at severity 1. If effective priorities change, rebuild the heap or remove and reinsert affected patients; mutating comparator keys while they remain in a heap can invalidate heap ordering. Clinical policy must define thresholds and be approved by clinicians. The submitted code implements the core priority rule; aging is a described deployment extension, not a clock-driven implementation.

8. Deployment decision: 5,000 arrivals/day

Five thousand arrivals over 24 hours averages approximately 208.3 arrivals per hour, or 0.058 arrivals per second. This daily average is not a peak-load guarantee. PriorityQueue is computationally suitable as a queue component at this scale, but deployment also depends on burst arrivals, service times, staffing, persistence, audit logs, concurrency, failover, privacy and clinical validation. The complete hospital system must be tested against peak arrival patterns and integrated with validated triage procedures.

9. Conclusion

A PriorityQueue with severity-ascending and arrival-ascending comparison implements the requested triage policy and preserves FIFO order among equal severities. A copied queue drained with poll() gives a correctly ordered waiting-list display. The alternatives trade admission and removal costs differently. Aging can reduce starvation but requires careful heap reordering and clinical validation.